OOPs Notes

OOPs (Object-Oriented Programming) is a programming approach based on classes and objects. It helps organize code and reuse it efficiently.

1. Class

A class is a user-defined data type that acts as a blueprint for creating objects. It contains data members (variables) and member functions (methods).

Example: A Student class can contain a student's name, roll number, and a function to display details.

2. Object

An object is an instance of a class. It has its own instance data and can access the class's public member functions.

Example:

```
Student s1;\nStudent s2;
```

Here, s1 and s2 are two separate objects of the Student class.

3. Constructor

A constructor is a special member function that is automatically called when an object is created. It is generally used to initialize an object's data members.

- Has the same name as the class.
- Has no return type, not even void.
- Is called automatically when an object is created.
- Can be overloaded.

Types of constructors

1. Default constructor: Takes no arguments.

2. Parameterized constructor: Takes arguments to initialize an object.

3. Copy constructor: Creates an object using another object of the same class.

Code example:

```
#include <iostream>
using namespace std;

class Student {
public:
  int age;

  // Default constructor
  Student() {
    age = 18;
  }
```

```
    // Parameterized constructor
    Student(int a) {
        age = a;
    }
};

int main() {
    Student s1;
    Student s2(20);

    cout << s1.age << endl;
    cout << s2.age << endl;

    return 0;
}
```
Output:\n18\n20

The constructors have the same name but different parameter lists. This is called constructor overloading.

4. Destructor

A destructor is a special member function automatically called when an object's lifetime ends. It is commonly used to release resources owned by an object.

- Has the same name as the class, preceded by ~.
- Has no return type.
- Takes no arguments.
- Cannot be overloaded.
- Is called automatically when an object is destroyed.

Code example:

```
#include <iostream>
using namespace std;

class Student {
public:
    Student() {
        cout << "Constructor called" << endl;
    }

    ~Student() {
        cout << "Destructor called" << endl;
    }
};

int main() {
    Student s1;
    cout << "Inside main" << endl;

    return 0;
}
```
Output:\nConstructor called\nInside main\nDestructor called

The destructor runs when s1 goes out of scope at the end of main().

5. Inheritance

Inheritance is an OOP concept in which one class acquires accessible properties and behaviors from another class. It promotes code reusability.

Base class / Parent class: The class whose features are inherited.

Derived class / Child class: The class that inherits those features.

Code example:

```
#include <iostream>
using namespace std;

class Animal {
public:
   void eat() {
      cout << "Animal eats" << endl;
   }
};

class Dog : public Animal {
public:
   void bark() {
      cout << "Dog barks" << endl;
   }
};

int main() {
   Dog d;

   d.eat();   // Inherited function
   d.bark();  // Own function

   return 0;
}
```

Output:\nAnimal eats\nDog barks

Here, Dog inherits the eat() function from Animal.

Types of inheritance

1. Single inheritance

One derived class inherits from one base class. Example: A → B.

2. Multilevel inheritance

A class inherits from a derived class. Example: A → B → C.

3. Multiple inheritance

One derived class inherits from multiple base classes. Example: A + B → C.

4. Hierarchical inheritance

Multiple derived classes inherit from one base class. Example: A → B, C.

5. Hybrid inheritance

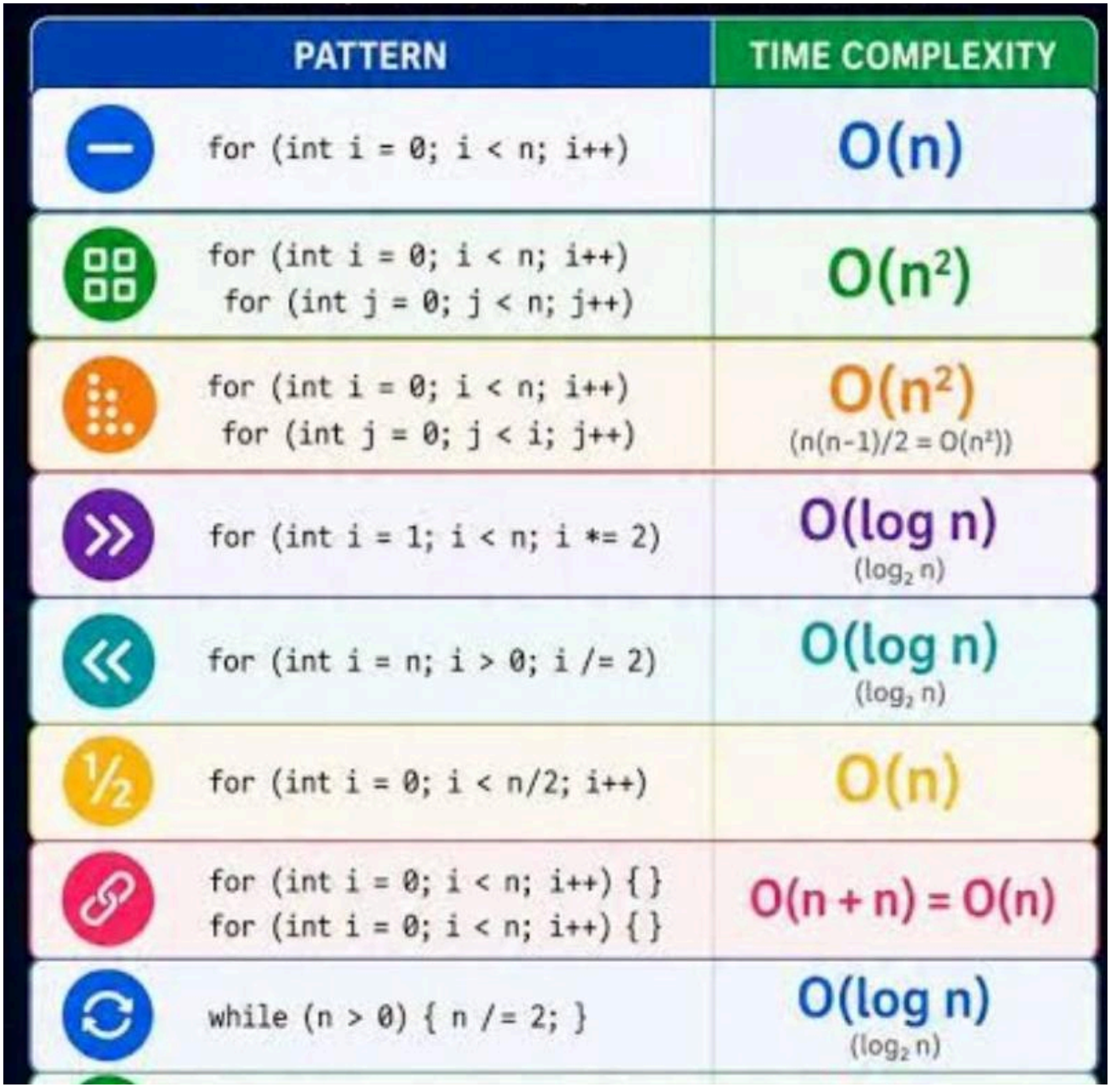
A combination of two or more inheritance types.

Key points to remember

A constructor is called when an object is initialized; a destructor is called when its lifetime ends. Inheritance allows a derived class to reuse accessible functionality of a base class.

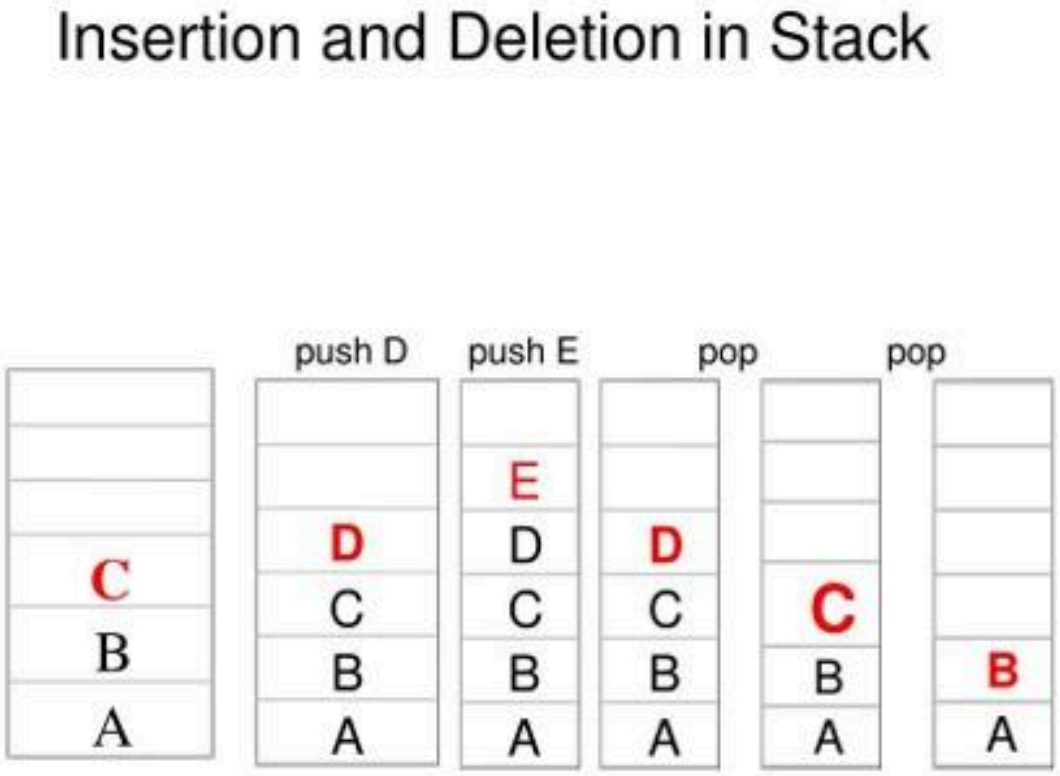
Access Modifiers in C++

Modifiers	Own Class	Derived Class	main()
Public	Yes	Yes	Yes
Private	Yes	No	No
Protected	Yes	Yes	No

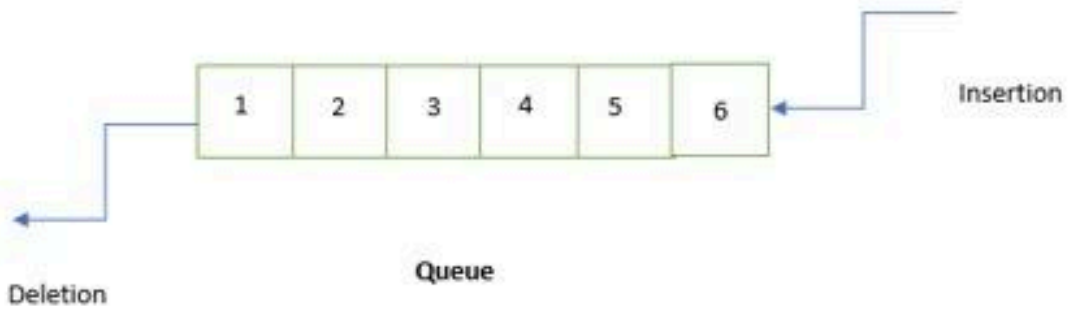


CONSTRAINTS	WORST CASE COMPLEXITY
N <= 10	O(N!), O(2^N)
N <= 18	O(2^N * N^2)
N <= 22	O(2^N * N)
N <= 10^2	O(N^4)
N <= 400	O(N^3)
N <= 2000	O(N^2 log N)
N <= 10^4	O(N^2)
N <= 10^6	O(NlogN)
N <= 10^8	O(N)

STACK


Insertion and Deletion in Stack
push D
push E
pop
pop
C
B
A
D
C
B
A
E
D
C
B
A
D
C
B
A
C
B
A
B
A

QUEUE


1
2
3
4
5
6
Insertion
Deletion
Queue

CODE STACK

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
    queue<int> q;

    q.push(10);  // Insert 10
    q.push(20);  // Insert 20
    q.push(30);  // Insert 30
// 20 30
    cout << q.front() << endl; // 10
    cout << q.back() << endl;  // 30

    q.pop();  // Remove 10

    cout << q.front() << endl; // 20
    cout << q.size() << endl;  // 2

    if (q.empty())
        cout << "Queue is empty";
    else
        cout << "Queue is not empty";

    return 0;
}
```

QUEUE

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<int> s(5);
    int size=0;

    s.push(10);
    size++;// Insert 10
    s.push(20);
    size++;// Insert 20
    s.push(30);  // Insert 30
// 10 20
    cout << s.top() << endl;  // 30

    s.pop();   // Remove 30

    cout << s.top() << endl;  // 20
    cout << s.size() << endl; // 2

    if (s.empty())
        cout << "Stack is empty";
    else
        cout << "Stack is not empty";

    return 0;
}
```